

Season 1: How LLMs work

Embeddings.

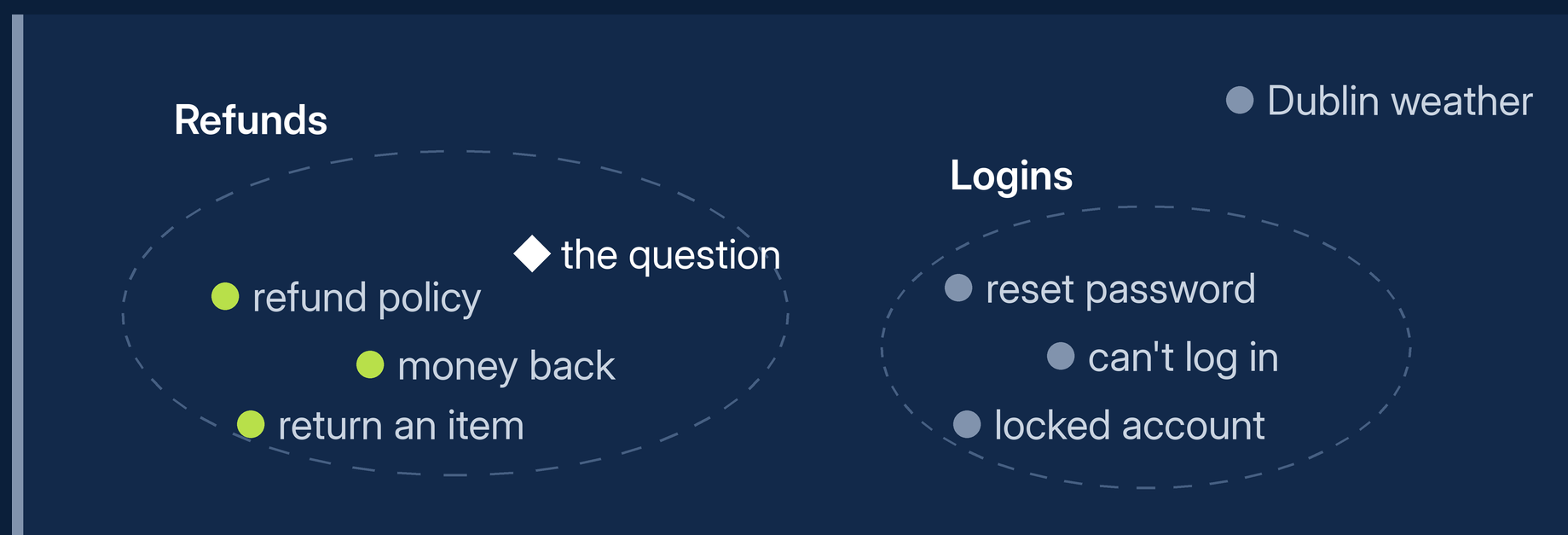
How models turn tokens into meaning they can compare

Meaning you can measure

- **Inside every model:** the first thing a model does with a token ID is look up its embedding
- **Search:** find text by what it means, not the words it uses
- **Retrieval:** RAG, in Season 2, is built on them
- **Grouping:** cluster, deduplicate, and classify without labels

An embedding places text on a map of meaning

An embedding model turns text into a list of numbers, 1,536 for OpenAI's text-embedding-3-small. Treat them as coordinates: texts with similar meaning land close together.

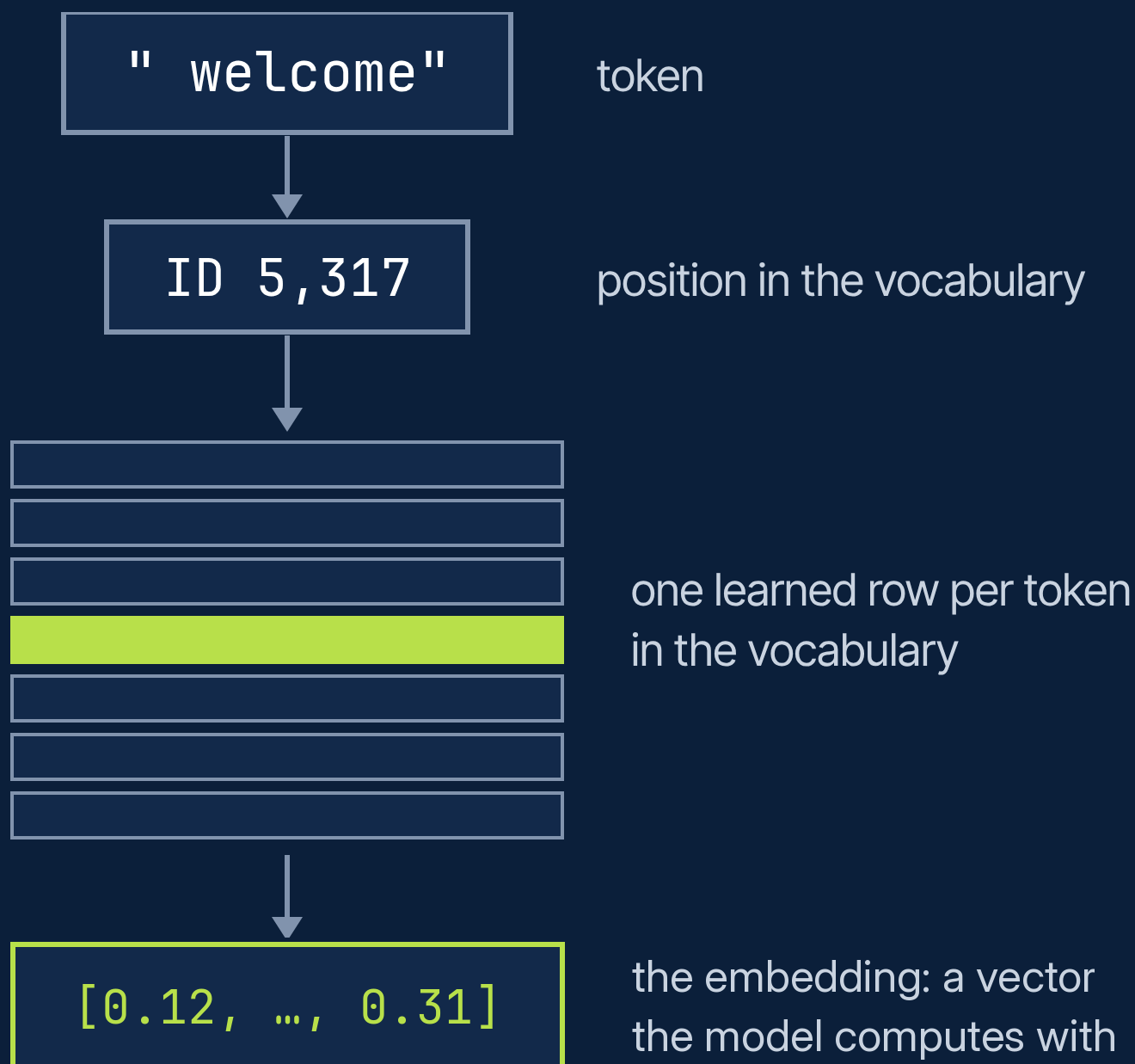


Illustrative, in 2 dimensions instead of 1,536. Search places the question ("How do I get my money back?") on the same map and returns its nearest neighbours.

Where it breaks

Near means similar wording and topic, not the same meaning. "The drug is safe" and "The drug is not safe" scored 0.807, closer than a true paraphrase at 0.592.

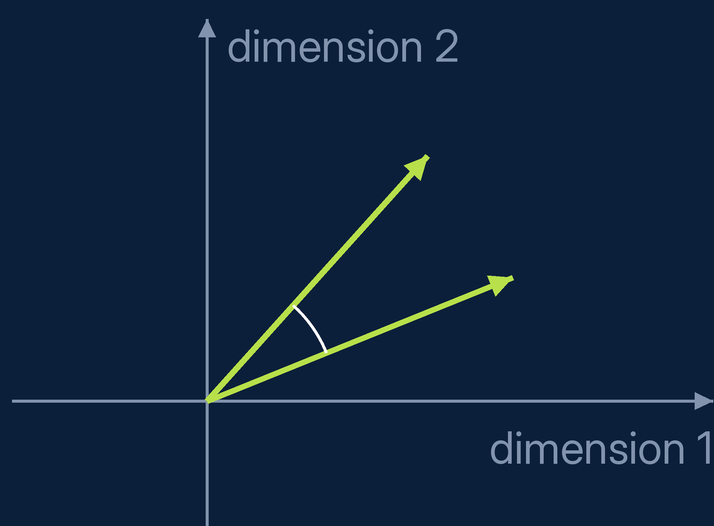
From token to vector



Illustrative ID and values. Every row is learned during training, and real rows hold hundreds to thousands of numbers rather than the handful shown.

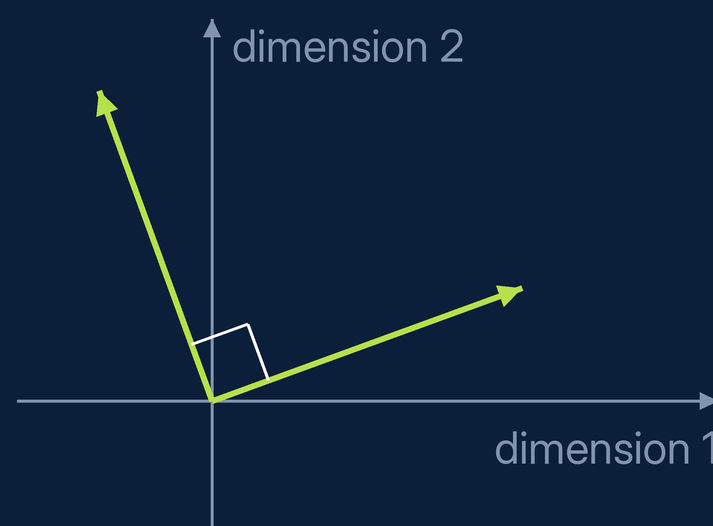
Closeness is measured by angle

Cosine similarity measures the angle between two embedding vectors. It ignores how long they are and asks one question: do they point the same way?



Small angle: score near 1

The texts are very similar



Right angle: score 0

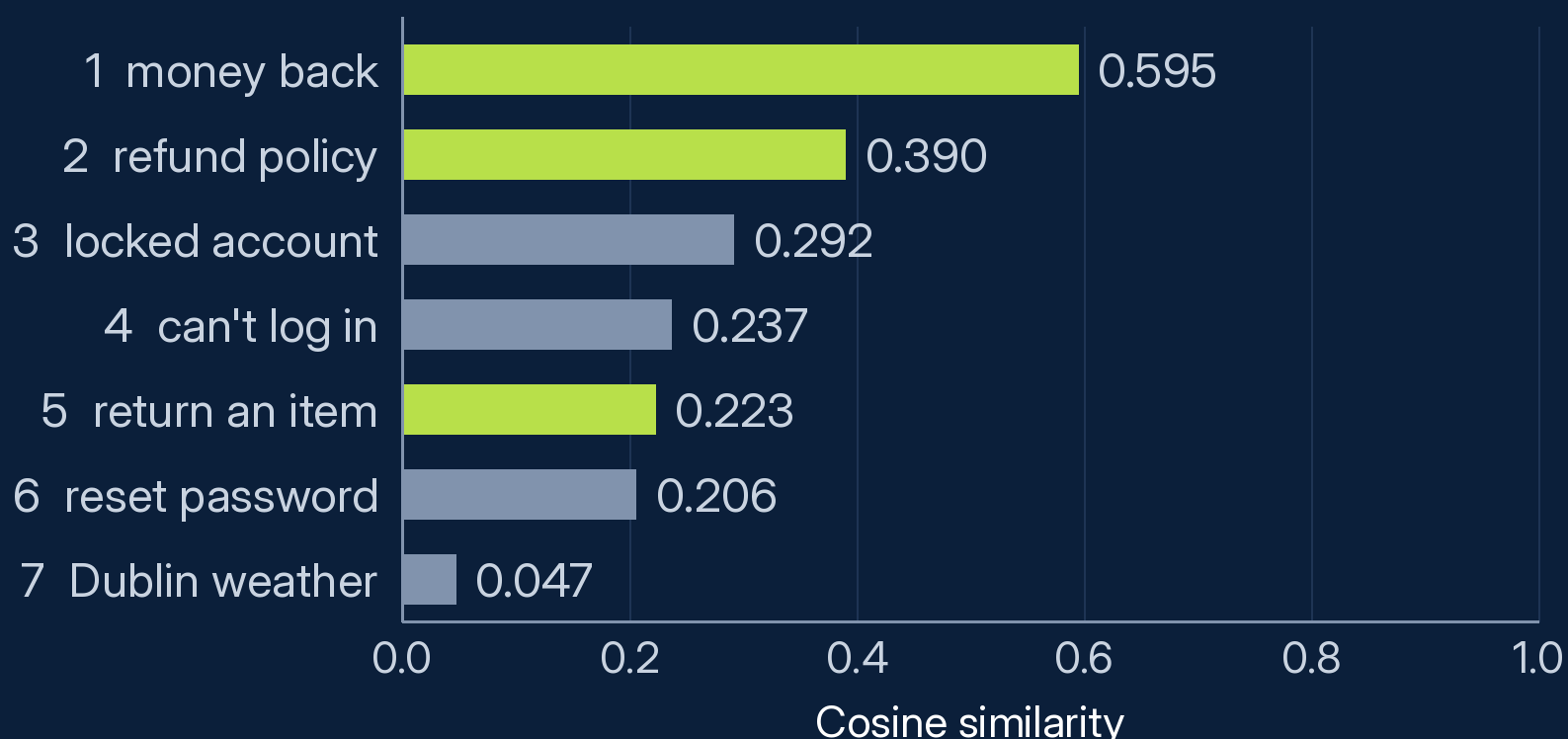
The texts are unrelated

How search uses this. The system embeds the question, scores every stored passage by its angle to the question, and returns the highest-scoring passages first.

Drawn in 2 dimensions for clarity; text-embedding-3-small uses 1,536. Real scores rarely reach 0: here, unrelated sentences still scored 0.104. Compare scores with each other, not against a fixed scale.

Search found two refund answers and missed the third

The test: One customer question, "How do I get my money back?", compared with seven stored phrases and ranked by score. Three phrases answer the question (green). Three are about logins. "Dublin weather" is a deliberately unrelated control, showing what a non-match scores.

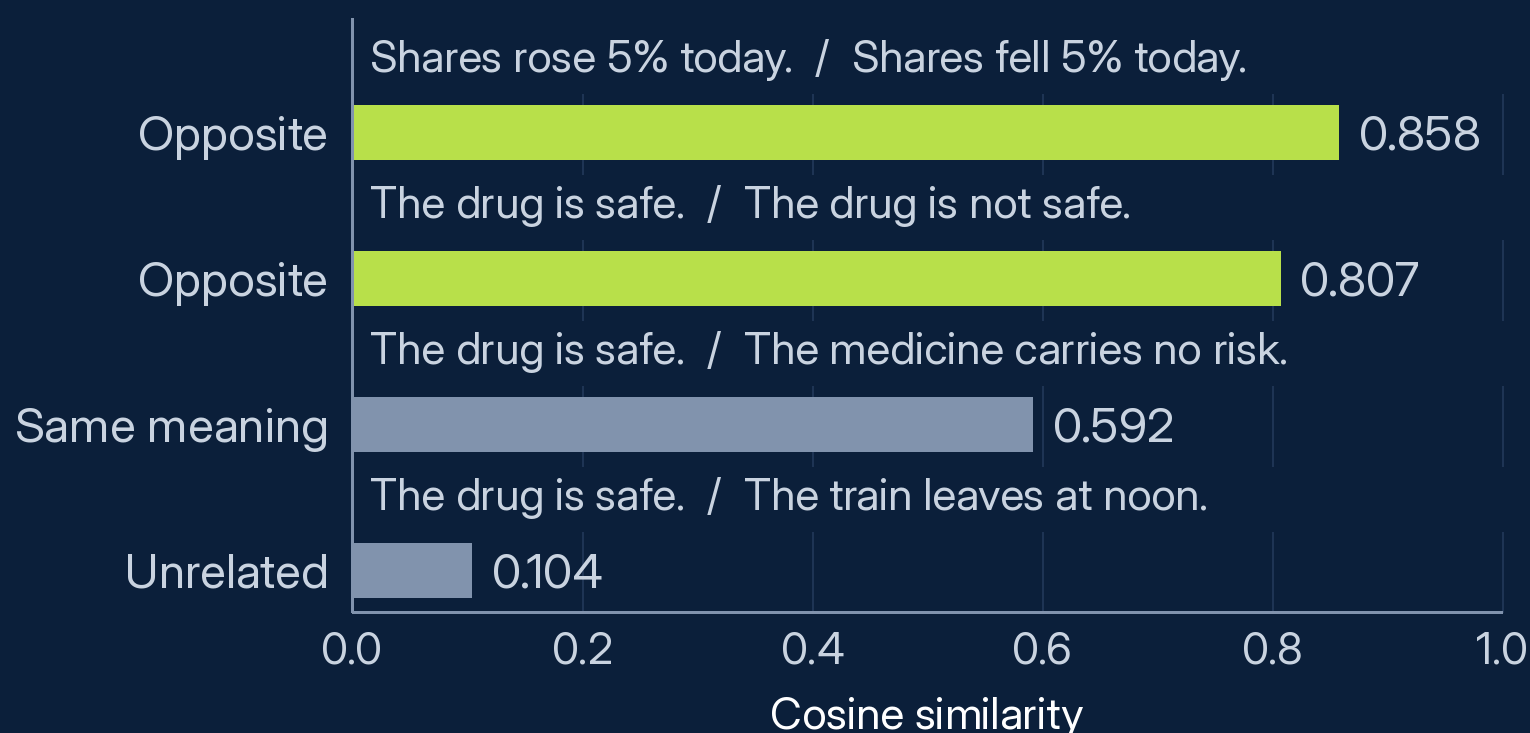


Reading it: "money back" came first because it repeats words from the question. "return an item" answers it too but shares no words with it, and ranked fifth, below two login problems.

text-embedding-3-small, 7 stored phrases, one run, 19 September 2026.

A sentence and its opposite scored closer than a paraphrase

The test: Four sentence pairs, each scored for closeness: near 1 means nearly the same, near 0 means unrelated. If the model captured meaning, the paraphrase would win.



Why: the model scores shared wording and topic, not truth. Each opposite pair differs by one word, so the vectors barely move. The paraphrase shares almost no words, so it lands further away despite meaning the same thing.

text-embedding-3-small, 4 pairs, one run, 19 September 2026.

Common kinds of embedding

Token embeddings

Inside the model: one vector per token, reshaped layer by layer as the model reads (S1 E4 and E5)

Text embeddings

From an embedding model: one vector for a whole passage, stored and searched (Season 2)

Where it breaks

Vectors from different models live in different spaces. They cannot be compared, even with matching dimensions.

Other kinds exist, such as image and multimodal embeddings. They rest on the same idea: one vector per item, compared by angle.

Similar is not the same as relevant

- **Opposites score as similar:** "The drug is safe" and "The drug is not safe" scored 0.807, above a true paraphrase at 0.592. One word reverses the meaning but barely moves the vector, so search can return text that says the opposite.
- **Direction gets lost:** "Shares rose 5% today" and "Shares fell 5% today" scored 0.858, the closest pair tested. Direction words carry little weight, so gains and losses blur together.
- **Same need, different words:** A refund question needs the returns article, yet "return an item" ranked fifth, below two login problems, as it shares no words with the question.
- **Exact terms match poorly:** Codes, IDs and names such as SKU-4471 carry little meaning for the model. Hybrid search adds keyword matching to catch them (Season 2).

Bullets 1 to 3 measured with text-embedding-3-small, one run, 19 September 2026.
Bullet 4 is general guidance.

Run it yourself

The code behind slide 6. One API call embeds all eight texts.

```
import numpy as np
from openai import OpenAI

client = OpenAI()
query = "How do I get my money back?"
phrases = ["refund policy", "money back", "return an item",
           "reset password", "can't log in", "locked account",
           "Dublin weather"]
result = client.embeddings.create(
    model="text-embedding-3-small", input=[query, *phrases])
vectors = np.array([item.embedding for item in result.data])
q, rows = vectors[0], vectors[1:]
norms = np.linalg.norm(rows, axis=1) * np.linalg.norm(q)
scores = rows @ q / norms
for score, phrase in sorted(zip(scores, phrases), reverse=True):
    print(f"{score:.3f}  {phrase}")
```

■ Cosine similarity: the dot product, divided by both lengths

[episodes/s1-e3-embeddings/similarity.py](#). Needs an OpenAI API key; the charts redraw from committed results without one.

Working with vectors

One model

Embed queries and documents with the same model and version. Vectors from different models cannot be compared.

Record it

Store the model name and version with every vector, so you know exactly what to re-embed when the model changes.

Normalise

Scale every vector to length 1 if the provider does not (OpenAI already does). Cosine similarity then gives the same result as a plain dot product, which vector databases compute faster.

Test on your data

Measure retrieval on your own queries before trusting a public leaderboard. Benchmark scores do not carry over to every domain.

What breaks in production

- **Model drift:** the embedding model is upgraded for new queries, but the index still holds vectors from the old model. Scores stop meaning anything, and nothing raises an error.
- **Mixed indexes:** vectors from two different models end up in one store. Their scores cannot be compared, so the ranking is effectively noise.
- **Guessed thresholds:** many systems keep only results above a fixed score, often picked without testing. Too high discards good answers; too low lets poor ones through. Here, a cut-off of 0.25 would keep "locked account" (0.292) and drop "return an item" (0.223).
- **Chunks that are too long:** one vector for a long passage blends many topics into a single point, so it matches none of them strongly. Chunking is covered in Season 2.

Changing embedding model is a migration, not a setting

A new model cannot search an index built by the old one.

Switching means:

- **Re-embed everything:** every stored document, not just new content
- **Rebuild and retune:** a new index, and new thresholds for the new model's scores
- **Re-test retrieval:** on your own queries, before switching over

Ask your team

Do we keep the source text for every vector, and what would it take to switch embedding model?

Decision rule

Treat the embedding model as a versioned dependency: pin it, test it on your own queries, and re-embed when it changes.

Next: S1 E4 Attention

How a model decides which earlier tokens matter for the next one.

Where this fits

The series map

S1 LLMs	S2 Retrieval	S3 Strategy
S4 Adapting	S5 Agents	S6 Economics
S7 Data	S8 Evaluation	S9 AI teams
S10 Reliability	S11 Security	S12 Delivery
S13 Performance	S14 Resilience	S15 Governance

☐ Leadership season

 Follow	linkedin.com/in/brendan-kelly-irl
 Episode guide	brendankellyai.github.io
 Code	github.com/BrendanKellyAI/applied-ai-lab